

ORIENTADA A OBJETOS (POO)

1. Evolución y Concepto de la POO

La POO surgió como respuesta a las limitaciones de la programación estructurada, la cual presentaba dificultades de mantenimiento en sistemas complejos.

- **Diferencia clave:** Mientras la programación estructurada se centra en acciones y funciones, la POO se enfoca en **entidades (objetos)** que interactúan entre sí.
- **Propósito:** Modelar el mundo real mediante objetos que combinan datos (**atributos**) y comportamientos (**métodos**).
- **Ventajas:** Ofrece modularidad, reutilización de código, facilidad de mantenimiento, abstracción y escalabilidad.

2. Elementos Básicos

- **Clase:** Es la plantilla o modelo que define los atributos y métodos comunes.
- **Objeto:** Es una instancia concreta de una clase (ej. "auto1" es un objeto de la clase "Vehículo").
- **Atributos:** Propiedades que describen el estado del objeto.
- **Métodos:** Operaciones o acciones que el objeto puede realizar.

3. Principios Fundamentales

1. **Encapsulamiento:** Agrupa datos y métodos en una unidad, protegiendo la información mediante modificadores de acceso (público, privado, protegido).
2. **Herencia:** Permite que una clase (subclase) herede atributos y métodos de otra (clase padre), facilitando la reutilización y extensión del código.
3. **Polimorfismo:** Permite que diferentes clases respondan de forma distinta a un mismo mensaje o método, ya sea por **sobrecarga** (mismo nombre, distintos parámetros) o **sobrescritura** (redefinir el método en la subclase).
4. **Abstracción:** Identifica los aspectos esenciales de un sistema ignorando los detalles irrelevantes, implementándose mediante **interfaces** y **clases abstractas**.

4. Principios de Diseño SOLID

Estos cinco principios buscan crear arquitecturas limpias y mantenibles:

- **SRP (Responsabilidad Única):** Una clase debe tener un solo propósito.
- **OCP (Abierto/Cerrado):** Abierto para extender, cerrado para modificar.
- **LSP (Sustitución de Liskov):** Las subclases deben ser sustituibles por sus clases base sin errores.

- **ISP (Segregación de Interfaces):** Es mejor tener interfaces pequeñas y específicas que una sola monolítica.
- **DIP (Inversión de Dependencias):** Los módulos de alto nivel deben depender de abstracciones, no de implementaciones concretas.

5. Modelado con UML

El **Lenguaje de Modelado Unificado (UML)** permite visualizar la estructura del sistema antes de programarlo.

- **Diagrama de Clases:** Representa la estructura estática (nombre, atributos, métodos) y las relaciones (asociación, agregación, composición y herencia).
- **Diagrama de Objetos:** Muestra instancias específicas en un momento determinado para validar el comportamiento del modelo.
- **Herramientas sugeridas:** StarUML, Visual Paradigm y Enterprise Architect.



Figura 1

Representación de la Programación Orientada a Objetos (POO).